

Scientific Workflows from Zero to Hero with DAGonStar

A Primer for AI--HPC Computational Environmental Science

Dante Domizzi Sánchez-Gallegos

dantsanc@pa.uc3m.es



What You Will Be Able to Do

By the end of the tutorial, you will be able to:

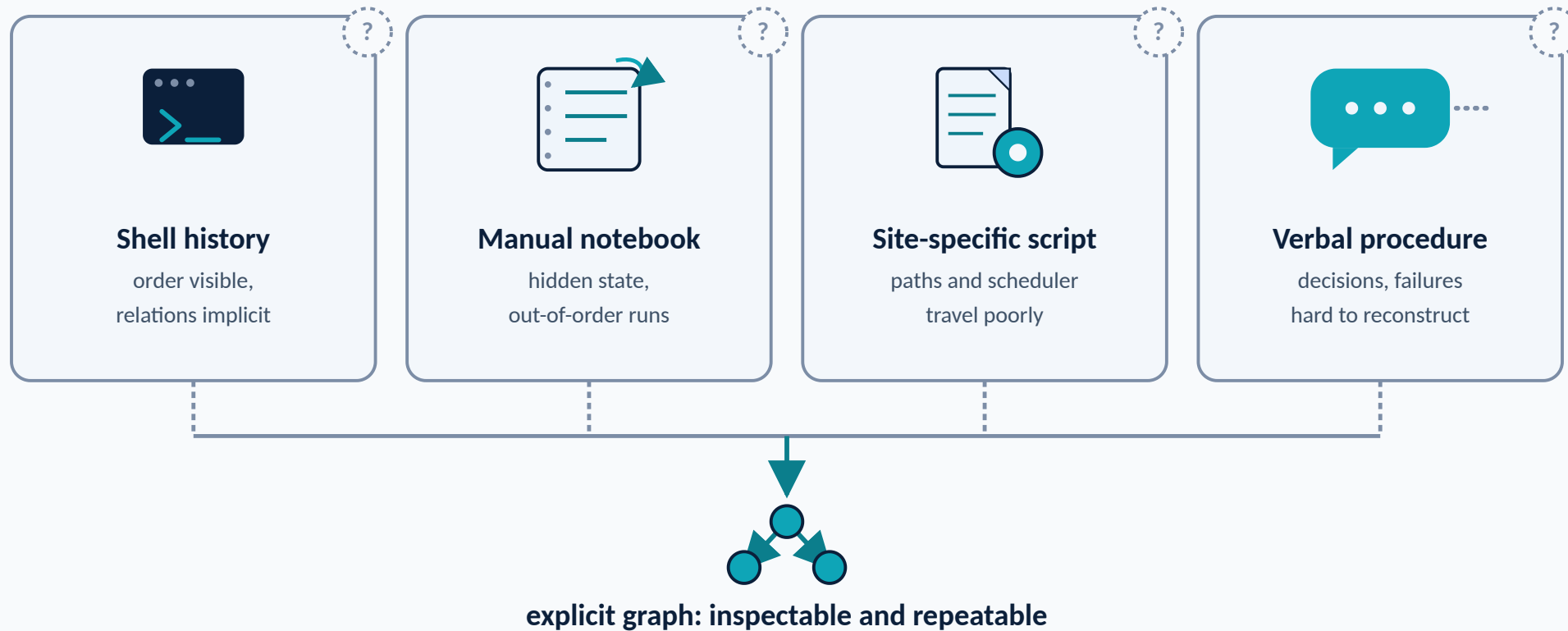
- identify tasks, inputs, outputs, and dependencies in a research pipeline;
- build and run a local DAGOnStar workflow;
- distinguish control dependencies from data-oriented dependencies;
- inspect scratch directories, launch evidence, staging, and checkpoints;
- reason about Docker, SSH, Slurm, Web, LLM, Native, and extended task boundaries;
- identify where FAIR recording, CWL export, and portable emulation fit;
- sketch a reliable AI--HPC workflow for your own research.

Agenda: Foundations and Local Execution

1. Workflow Primer: What Scientific Workflows Are and Why They Matter
2. DAGonStar: The Computational Model
3. Setup and First Local Execution
4. Dependencies and Provenance: Explicit Edges, Data-Oriented Edges, `workflow://`, Scratch Directories, and Launchers
5. Making Workflows Reliable: Validation, Cycle Detection, Checkpointing, and Staging
6. Running Beyond the Laptop: Containers, Remote Execution, Slurm, and External Scientific Software
7. AI--HPC Workflow Steps: LLM Tasks, Web Tasks, and Native Python Tasks
8. FAIR by Design: Provenance Recording as a Byproduct of the Graph

1. Workflow Primer: What Scientific Workflows Are and Why They Matter

Why Command History Is Not Enough



Each method hides something -- intent, state, assumptions, or decisions -- behind the commands actually run. A workflow makes that same structure inspectable and repeatable.

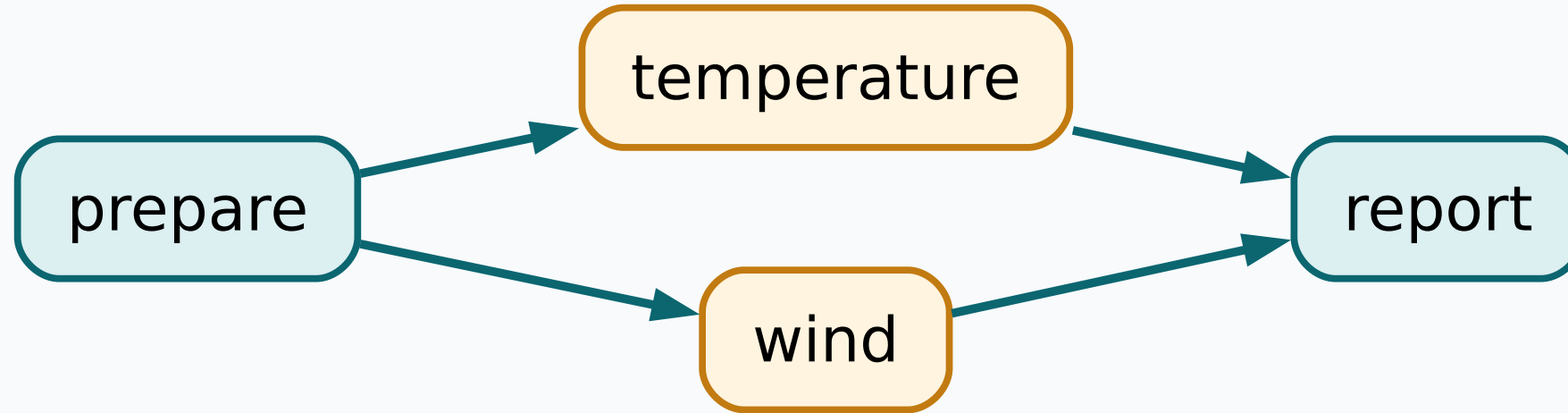
Anatomy of a Scientific Pipeline

For every step, identify:



These six answers become the raw material of a workflow graph.

Tasks and Dependencies



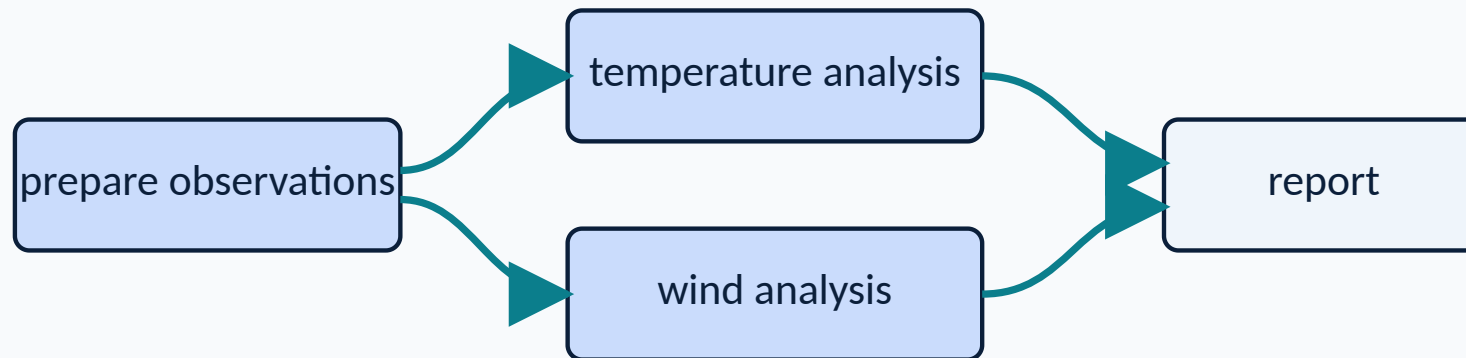
- A **task** is one managed unit of work.
- A **dependency** states that one task requires another.
- **Fan-out** exposes independent work after a shared producer.
- **Fan-in** joins multiple results into one downstream product.
- Independent branches may execute concurrently.

Directed Acyclic Graph

DAG: Directed edges encode dependency order, and the graph contains no path that returns to its starting task.

Direction answers: **what must precede what?**

Acyclicity guarantees that at least one valid starting point exists.



A producer can fan out into independent analyses; their products can fan in to a downstream report.

What Workflow Systems Make Explicit

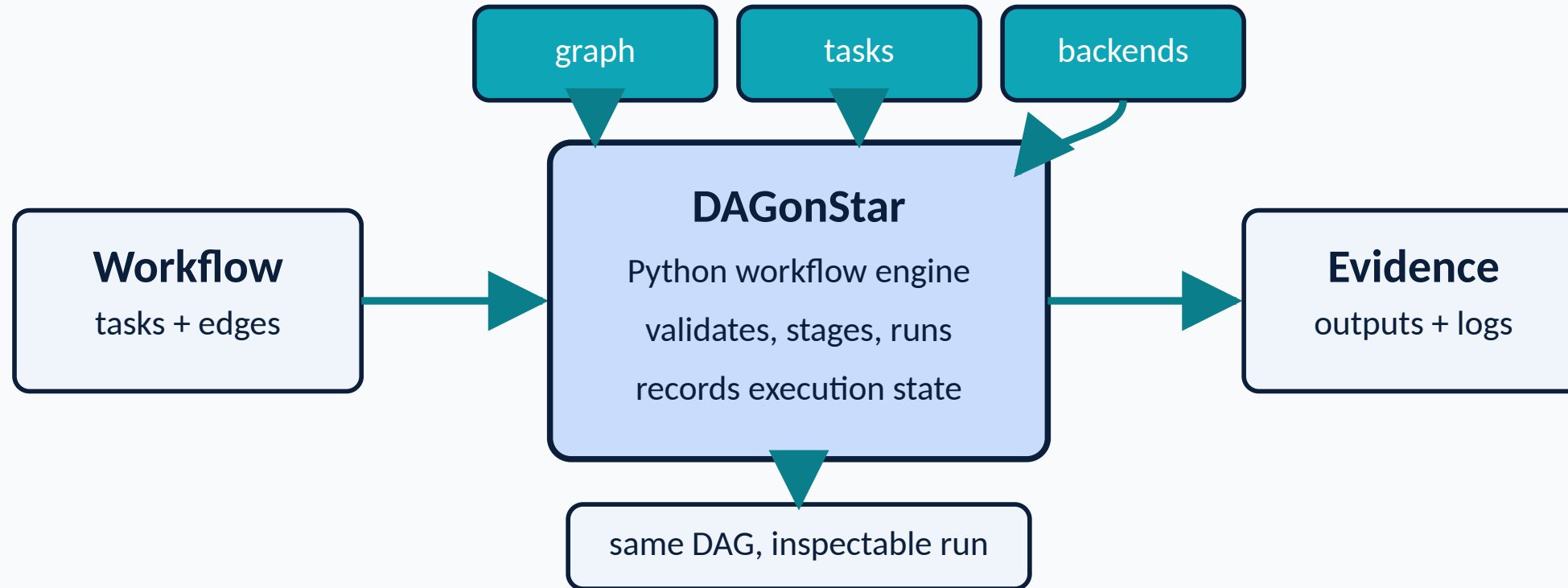
Concept	Practical value
Tasks and edges	Computational intent
Inputs and outputs	Data lineage
Parallel branches	Available concurrency
Runtime records	Provenance and diagnosis
Backend declaration	Execution assumptions
Checkpoints	Recovery after interruption
Portable graph model	A basis for moving across environments

Portability has boundaries: software, data, credentials, and site policy still need explicit treatment.

2. DAGonStar

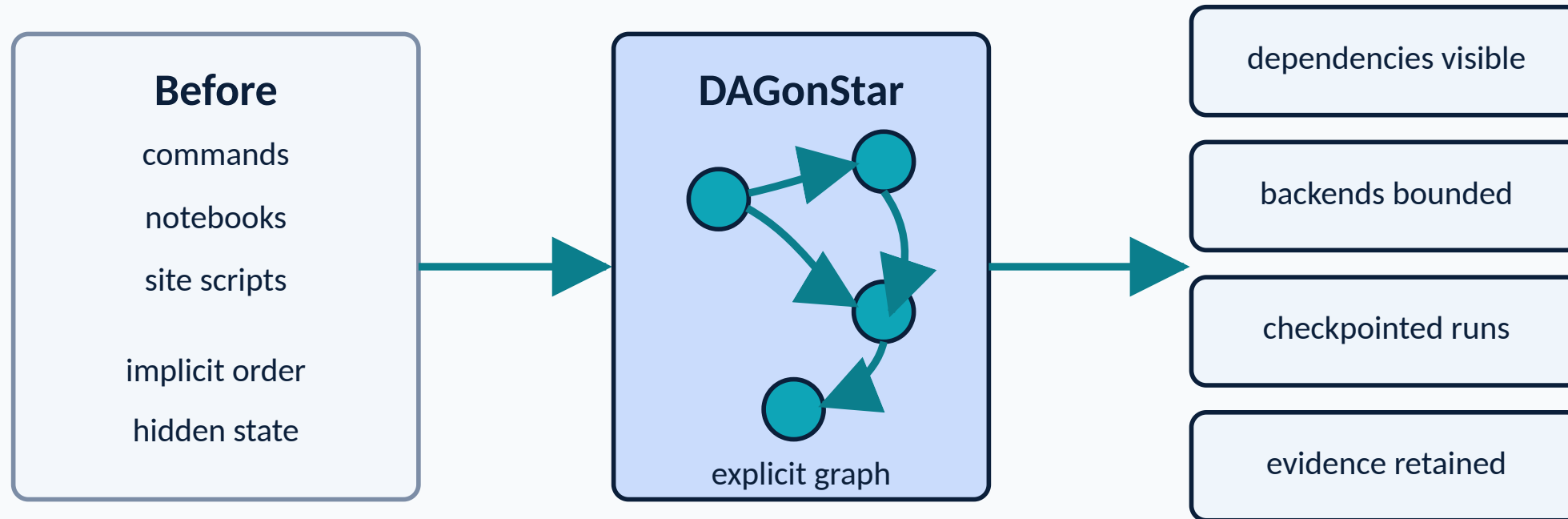


What DAGonStar Is



A Python workflow engine that turns tasks, dependencies, backends, and runtime evidence into an inspectable DAG run.

Why DAGonStar



The point is not automation for its own sake: the point is a workflow whose computational claims can be checked.

Developing Team



Main developer

**Dante Domizzi
Sánchez-Gallegos**

Universidad Carlos III de
Madrid



Main developer

**Ciro Giuseppe De
Vita**

University of Naples
"Parthenope"



Main developer

Raffaele Montella

University of Naples
"Parthenope"

CONTRIBUTORS

Juan Armando Barrón Lugo

CENIDET, México

Genaro Juan Sánchez-Gallegos

Universidad Carlos III de Madrid

**Catherine Alessandra Torres
Charles**

Universidad Carlos III de Madrid

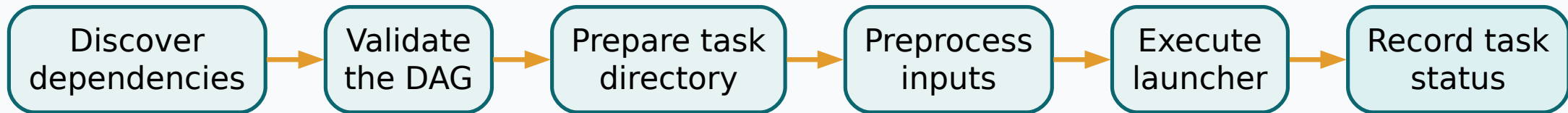
Four Important abstractions

Term	Working definition
<code>Workflow</code>	Named collection of tasks and dependency edges
<code>DagonTask</code>	One managed unit of work
<code>TaskType</code>	Execution model selected for that unit
<code>workflow://</code>	Logical reference to a producer artifact

```
from dagon import Workflow
from dagon.task import DagonTask, TaskType

wf = Workflow("FirstDAG", config=local_config(scratch))
```

What DagOnStar Coordinates



The shell command creates the scientific artifact.

DAGOnStar discovers dependencies, validates the graph, prepares task directories and inputs, invokes the selected task adapter, and records execution state.

3. Setup and First Local Execution

Local Setup Check

Use Python 3.8 or newer.

```
python3 -m venv .venv  
. .venv/bin/activate  
pip install git+https://github.com/DagOnStar/dagonstar.git
```

Optional integrations are available as install extras:

```
pip install "dagonstar[all]@git+https://github.com/DagOnStar/dagonstar.git"
```

Confirm the same interpreter can import DAGOnStar:

```
python -c "import dagon; print(dagon.Workflow.__name__)"
```

Tutorial Materials on Google Collab

Open the course notebook in Google Colab

<https://tinyurl.com/4hc3fne9>



Scan for Colab

Run the Notebook in VS Code

This repository also ships its own notebook, `DAGonStar_Tutorial.ipynb`, covering all 15 runnable lessons in one place, in the order this talk presents them.

1. Open the repository root folder in VS Code (the folder containing `tutorial/`).
2. Install the **Python** and **Jupyter** extensions if you do not already have them.
3. Open `DAGonStar_Tutorial.ipynb`.
4. Click the kernel picker (top right) and select the `.venv` interpreter you created on the Local Setup Check slide.
5. Run All. The first cell installs DAGonStar; the rest run every lesson in order.

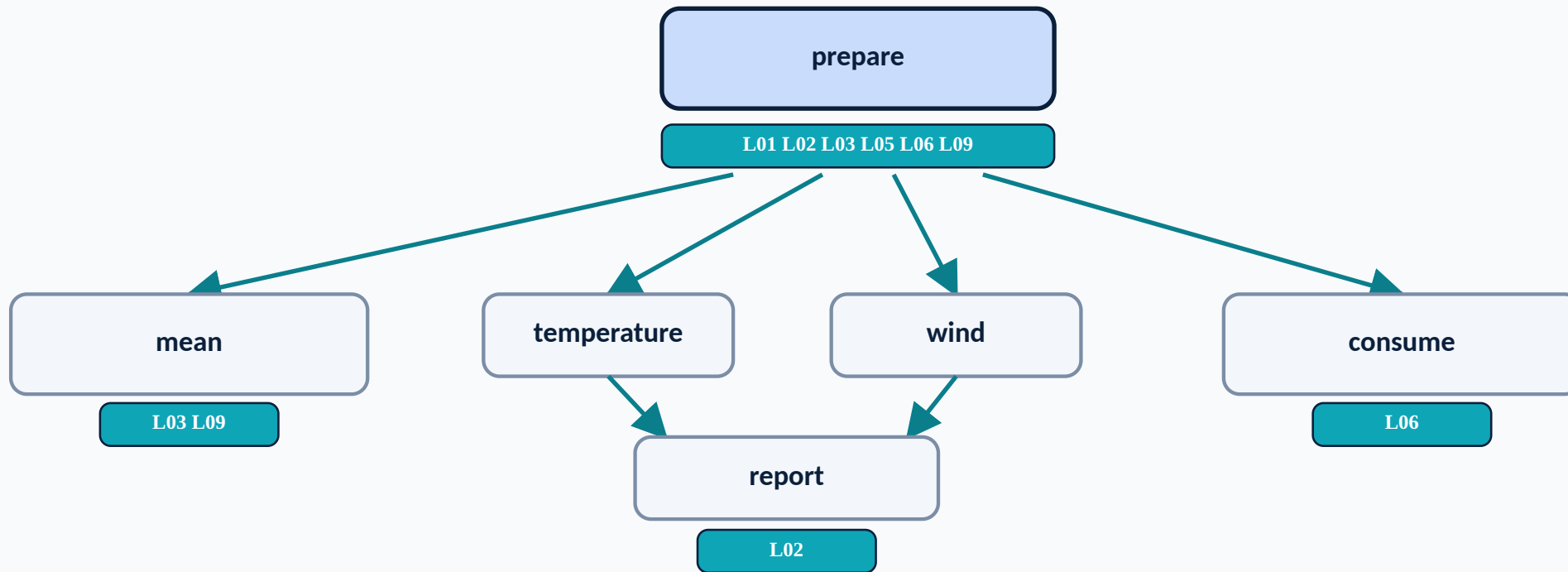
The notebook must be opened from the repository root -- it checks that `tutorial/` sits right next to it, the same requirement as running the scripts directly.

Setup Exercise Checkpoint

Complete one path:

1. Run the local import check, or open the fallback environment.
2. Identify the Python version and repository revision.
3. Locate `tutorial/` in this repository.
4. Confirm that the running scenario data is visible.

The Tutorial's Running Example



Build a Minimal Two-Task DAG

```
prepare = DagonTask(  
    TaskType.BATCH,  
    "prepare",  
    observation_command(),  
)  
  
report = DagonTask(  
    TaskType.BATCH,  
    "report",  
    "mkdir -p outputs; printf complete > outputs/status.txt",  
)  
  
wf.add_task(prepare)  
wf.add_task(report)  
report.add_dependency_to(prepare)
```

Read `report.add_dependency_to(prepare)` as: **report depends on prepare.**

Validate, Run, and Inspect

```
wf.Validate_WF()
wf.run()

result = Path(report.working_dir, "outputs", "status.txt")
assert result.read_text() == "complete"

print("prepare:", prepare.working_dir)
print("report:", report.working_dir)
```

Each task receives its own managed working directory. The output assertion checks scientific content separately from task completion.

Run Lesson 01: First Local Task

```
python tutorial/lesson_01_first_local_task.py
```

The task creates a deterministic CSV and DAGonStar verifies its content exactly. Run it now, before moving on.

Run Lesson 02: Build a DAG

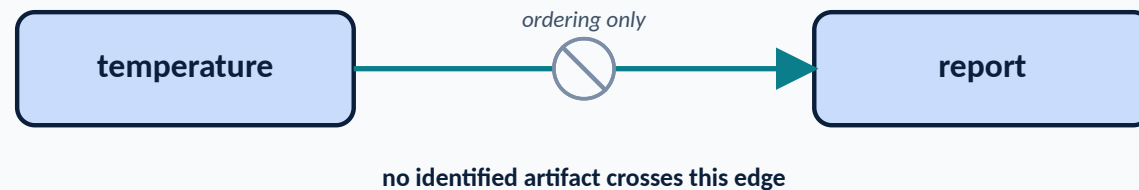
```
python tutorial/lesson_02_build_a_dag.py
```

The DAG reports `temperature` and `wind` as predecessors of `report`. Both scripts exit successfully after exact assertions, with task outputs under managed scratch directories. Run it now, before the checkpoint.

4. Dependencies and Provenance: Explicit Edges, Data-Oriented Edges, **workflow://**, Scratch Directories, and Launchers

Explicit Control Dependency

```
report.add_dependency_to(temperature)
```



This means **temperature** must finish before **report** starts.

It does **not** identify a file, copy an output, or validate scientific meaning.

Control dependency: an ordering relation without an identified artifact.

Use explicit edges when ordering is real but no file reference expresses it.

Data-Oriented Dependency

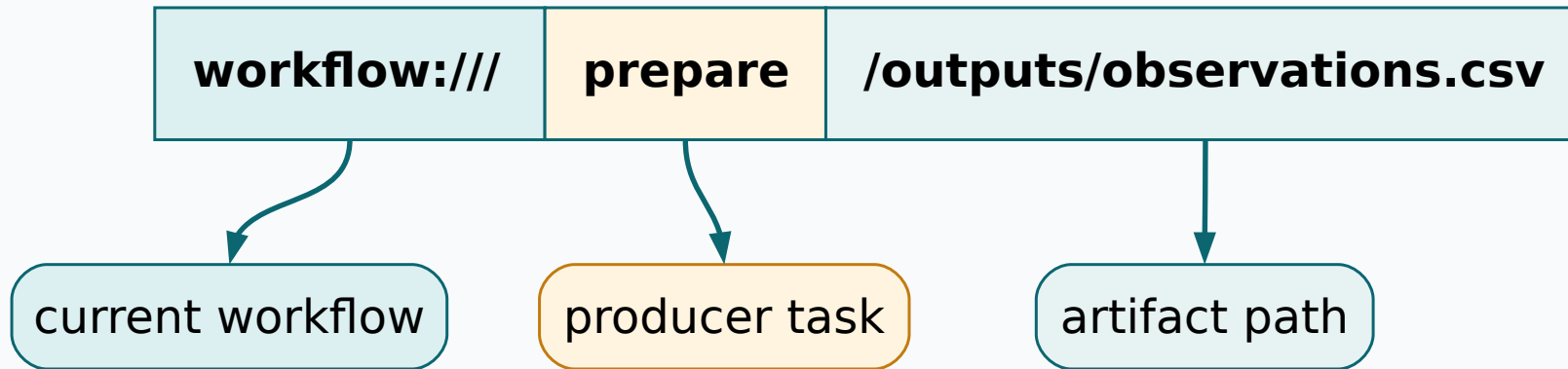
```
mean = DagonTask(  
    TaskType.BATCH,  
    "mean",  
    "mkdir -p outputs; "  
    "awk -F, 'NR>1{s+=$3;n++}END{print s/n}' "  
    "workflow:///prepare/outputs/observations.csv "  
    "> outputs/mean.txt",  
)
```

The command names:

- producer task: `prepare`;
- artifact: `outputs/observations.csv`;
- consumer task: `mean`.

The data reference also induces the required execution edge.

Reading **workflow://**



Same workflow:

```
workflow:///<task>/<artifact-path>
```

External workflow:

```
workflow://<workflow>/<task>/<artifact-path>
```

The URI records logical producer and artifact identity before a physical scratch path exists.

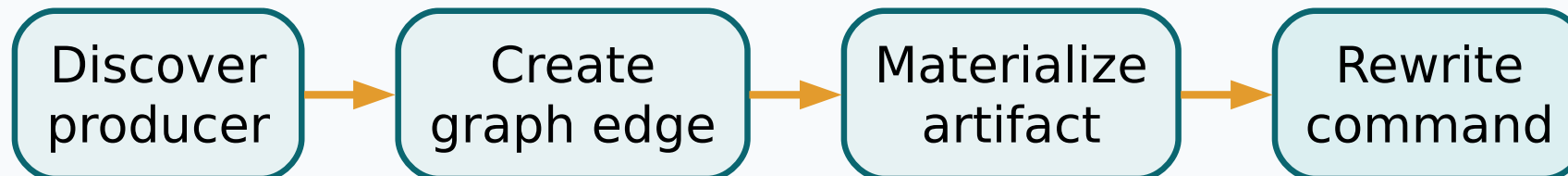
Dependency Discovery

```
wf.add_task(prepare)
wf.add_task(mean)

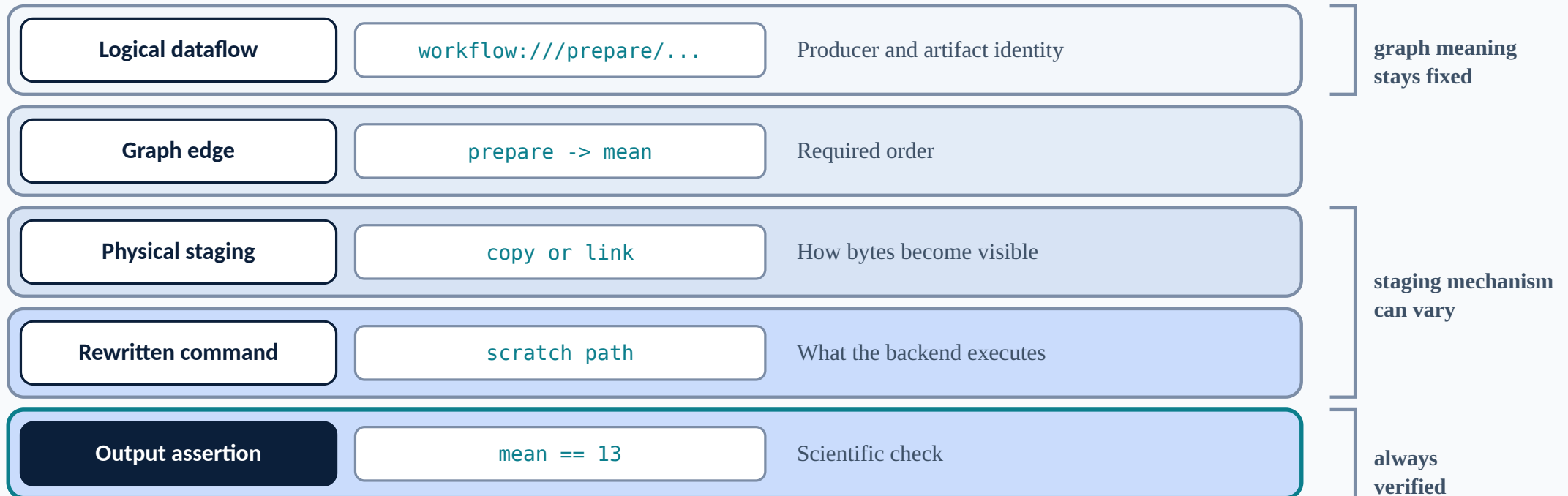
print("logical:", mean.command)
wf.make_dependencies()

assert [task.name for task in mean.prevs] == ["prepare"]
wf.run()
```

At execution time, DAGOnStar:



Keep the Layers Separate



Scratch Directories and Launch Evidence

```
wf.run()

task_dir = Path(task.working_dir)
scientific_output = task_dir / "outputs/observations.csv"
runtime_metadata = task_dir / ".dagon"

assert scientific_output.is_file()
assert runtime_metadata.is_dir()
```

Inspect the location reported by `task.working_dir` rather than guessing a `/tmp` path.

```
python tutorial/lesson_05_scratch_launchers_and_logs.py
```

Scientific Output vs Runtime Evidence

Evidence	Typical location	Question answered
Dataset or model result	<code>outputs/</code>	What did the computation produce?
Launcher or command record	<code>.dagon/</code>	What was invoked?
Standard output and error	task metadata/logs	What did the process report?
Task state	workflow/task metadata	Did the engine mark it complete?
<code>workflow://</code> reference	workflow definition	Which producer artifact was consumed?

No single artifact answers every provenance question.

Run the Data-Driven Example

```
python tutorial/lesson_03_workflow_data_dependencies.py
```

```
logical command: ... workflow:///prepare/outputs/observations.csv ...  
staged mean: 13 in <managed-task-directory>
```

Trace this chain:

```
logical reference -> inferred edge -> staged file -> rewritten command -> mean.txt
```

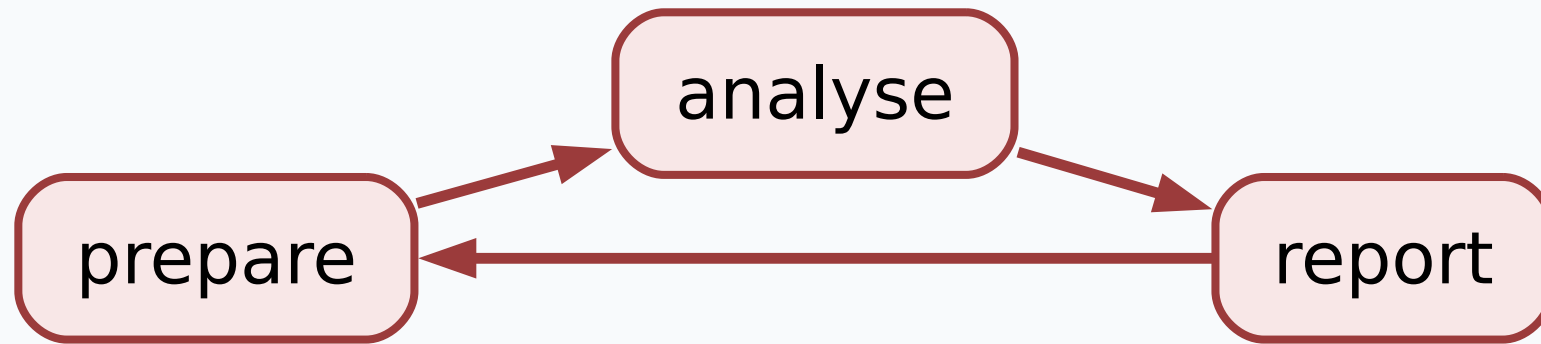
Dependencies and Provenance Checkpoint

For the `prepare -> mean` workflow, record:

1. The explicit task names.
2. The logical `workflow://` reference.
3. The inferred predecessor list.
4. The physical staged path.
5. The rewritten command evidence.
6. The verified output value.

5. Making Workflows Reliable: Validation, Cycle Detection, Checkpointing, and Staging

Why Cycles Fail



Cycle: a dependency path that returns to its starting task.

```

analyse.add_dependency_to(prepare)
report.add_dependency_to(analyse)
prepare.add_dependency_to(report)

wf.Validate_WF() # raises: cycle detected
  
```

No task on the cycle can have a valid first position.

Repair the Causal Model

The controlled example removes the incorrect edge and validates again:

```
prepare.prevs.remove(report)
report.nexts.remove(prepare)

wf.Validate_WF()
print("repaired graph validates")
```

```
python tutorial/lesson_04_validate_and_fix_cycles.py
```

In production code, repair the workflow definition. Direct list mutation is used here only to expose the validation experiment.

Staging Policy

```
from dagon import DataMover

wf.set_data_mover(DataMover.COPY)
```

or:

```
wf.set_data_mover(DataMover.LINK)
```

Mode	Behavior	Main trade-off
COPY	Creates an independent staged file	More storage and transfer
LINK	Creates a symbolic link	Shared filesystem required

Hold the Graph Constant

```
consumer = DagonTask(  
    TaskType.BATCH,  
    "consume",  
    "cat workflow:///prepare/outputs/observations.csv "  
    "> consumed.csv",  
)
```

Run the same producer, consumer, and `workflow://` edge under both policies:

```
python tutorial/lesson_06_data_staging.py
```

Expected observation: `COPY` yields a regular staged file; `LINK` yields a symbolic link.

Checkpoint and Resume

```
checkpoint = Path(scratch, "checkpoint.json")

wf = workflow(
    "Meteorology",
    scratch,
    checkpoint_file=str(checkpoint),
)
wf.add_task(calculate)
wf.run()
```

The first run writes completion state and records the task working location.

Resume Is an Explicit Decision

```
resumed = workflow(  
    "Meteorology",  
    scratch,  
    checkpoint_file=str(checkpoint),  
)  
resumed.add_task(same_calculation)  
resumed.run(resume_checkpoint_file=str(checkpoint))
```

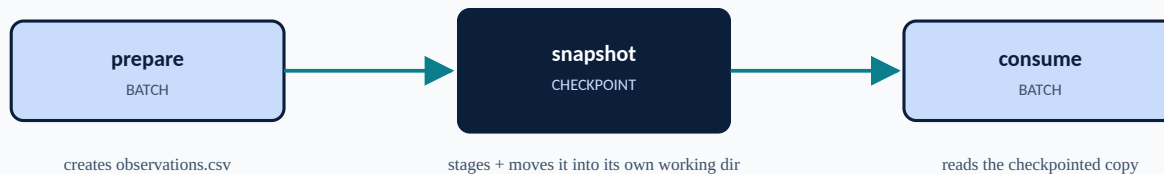
```
python tutorial/lesson_07_checkpoint_and_resume.py
```

Correctness condition: reuse is valid only when code, inputs, environment, and intended semantics still justify the saved result.

A checkpoint records execution state; it does not independently prove that undeclared external inputs are unchanged.

Explicit Checkpoint Tasks

```
DagonTask(TaskType.CHECKPOINT, "snapshot", "workflow:///prepare/outputs/observations.csv")
```



A different mechanism from the resume above: `TaskType.CHECKPOINT` stages the referenced file like any `workflow://` consumer, then moves it into its own working directory -- a stable, named copy downstream tasks depend on instead of the original producer.

```
python tutorial/lesson_14_explicit_checkpoint_task.py
```

Launch Without Blocking

```
wf.add_listener("on_workflow_start", lambda _: events.append("workflow_start"))
wf.add_listener("on_workflow_end", lambda _: events.append("workflow_end"))

thread = wf.launch()
assert wf.wait(5)
```

```
python tutorial/lesson_08_asynchronous_execution.py
```

`launch()` starts the workflow on a background thread and returns immediately, instead of blocking like `run()` does everywhere else in this tutorial. Listeners observe lifecycle events while it runs; `wait()` blocks -- with a timeout -- until it finishes.

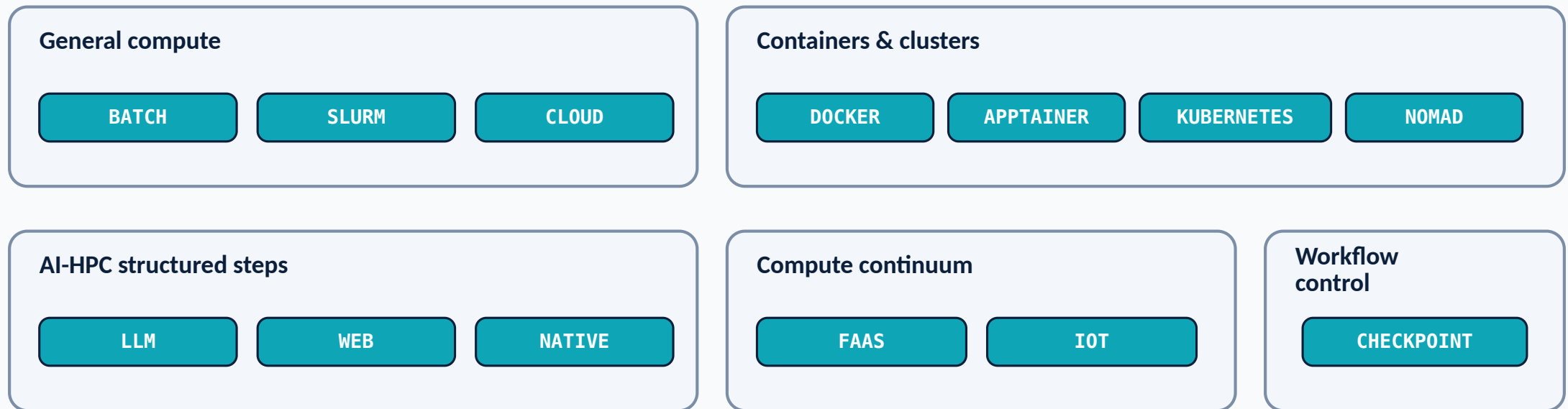
Reliability Checkpoint

For a failed or interrupted workflow:

1. Validate the graph before execution.
2. Separate dependency errors from task failures.
3. Choose staging according to filesystem and isolation needs.
4. Inspect task output and runtime evidence separately.
5. Launch asynchronously only when you need to keep working while it runs.
6. Resume only when the recorded state remains scientifically valid.
7. Reach for an explicit checkpoint task when a downstream consumer needs a stable, named snapshot rather than the whole workflow's resume state.

6. Running Beyond the Laptop: Containers, Remote Execution, Slurm, and External Scientific Software

All DAGonStar Task Types



DagonTask(TaskType.____, name, command, ...)

Backend Changes, Causality Does Not

```
DagonTask(TaskType.BATCH, "analyse", command)
DagonTask(TaskType.DOCKER, "analyse", command, image=image)
DagonTask(TaskType.SLURM, "analyse", command, partition=queue)
```

The backend may change:

- software environment;
- host and filesystem boundary;
- staging mechanism;
- scheduling and resource policy;
- available runtime evidence.

The dependency still means: **the consumer waits for its producer.**

Docker-Backed Task

```
mean = DagonTask(TaskType.DOCKER, "mean", command, image="alpine:latest")
```

Same `workflow:///` staging as `BATCH`; only where the command executes changes. Record exact image identity, platform, mounts, and resource limits -- for archival work, prefer an immutable digest over a mutable tag.

```
python tutorial/lesson_16_docker_backed_task.py
```

Run it now: this needs a real Docker daemon on this machine -- unlike every earlier lesson, it is not a mock.

Remote Batch Task over SSH

```
remote = DagonTask(TaskType.BATCH, "mean", command, ip=remote_host, ssh_username=remote_user, keypath=key_path)
```

Remote batch is selected by passing SSH parameters to the ordinary `BATCH` `DagonTask` factory -- there is no separate `TaskType.SSH`. Same `workflow:///` staging as `BATCH` and `DOCKER`; only where the command executes changes.

Credentials belong in runtime configuration, never in slides, source, logs, or exported provenance.

```
python tutorial/lesson_17_remote_batch_task_ssh.py
```

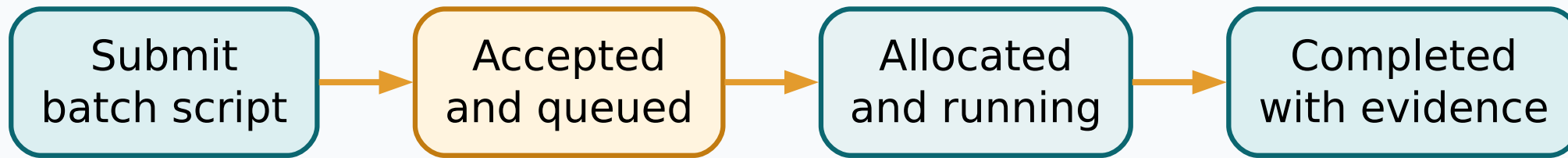
Needs a reachable SSH host and an authorized key -- see `tutorial/README.md`.

Slurm Task

```
job = DagonTask(  
    TaskType.SLURM,  
    "analysis",  
    "python run.py",  
    partition="debug",  
    ntasks=4,  
    memory=2048,  
    time="01:30:00",  
)
```

Site policy defines valid partitions and resource values. Submission, queueing, execution, and completion are different events.

Observe the Scheduler Boundary



Record:

- generated batch script and submission command;
- scheduler job identifier;
- requested and observed resources;
- queue transitions;
- final exit evidence and logs.

Local command-generation tests do not prove that a cluster accepted or completed a job.

External Scientific Software: The Practical Rule

1. **Verify the program first.** Run it directly with a small known input.
2. **Wrap it defensively.** Check executable, input, exit status, and output.
3. **Connect it last.** Add `workflow://` inputs and downstream dependencies.

This order separates:

- scientific-program failures;
- environment and licence failures;
- workflow staging failures;
- scheduler or transport failures.

Defensive Wrapper Pattern

```
#!/usr/bin/env bash
set -euo pipefail

input=$1
output=$2

command -v scientific_model >/dev/null 2>&1
test -r "$input"
mkdir -p "$(dirname "$output")"

scientific_model --input "$input" --output "$output"
test -s "$output"
```

Package or stage the wrapper with the workflow, then test it with a deterministic small case before selecting Docker, SSH, or Slurm.

Infrastructure Exercise

Start with:

```
local = DagonTask(TaskType.BATCH, "simulate", command)
```

Choose one target and identify what must be added:

Target	New declaration	New evidence
Docker	Image and mounts	Image identity and container result
SSH host	Host, user, key source, scratch	Trust, transfer, and remote result
Slurm	Partition and resources	Job ID, queue state, logs, and exit

7. AI--HPC Workflow Steps: LLM Tasks, Web Tasks, and Native Python Tasks

One Graph, Several Interaction Models

```
prepare observations
  |
  v
Native Python analysis
  |
  v
mock-backed LLM interpretation
  |
  v
local Web publication
```

The task type changes the execution adapter, not the need for dependencies, outputs, evidence, and failure handling.

Mock-Backed LLM Task

```
summarize = DagonTask(  
    TaskType.LLM,  
    "summarize_report",  
    {  
        "messages": [{  
            "role": "user",  
            "content": "Summarize this report: {report}",  
        }]  
    },  
    provider="local_mock",  
    input_files={  
        "report": "workflow:///prepare_report/output/report.txt",  
    },  
)
```

The local mock makes request construction, staging, and response capture testable without a paid API.

```
python tutorial/lesson_11_llm_tasks.py
```

Web Task

```
upload = DagonTask(  
    TaskType.WEB,  
    "upload",  
    {  
        "method": "POST",  
        "url": local_endpoint,  
        "multipart": {  
            "dataset": {  
                "file": "workflow:///prepare/output/data.txt",  
                "content_type": "text/plain",  
            }  
        },  
        "expected_status": 200,  
        "outputs": {  
            "body": "reply.json",  
            "metadata": "request.json",  
        },  
    },  
)
```

Native Python Task

```
native = DagonTask(  
    TaskType.NATIVE,  
    "mean",  
    "tutorial.common.native_functions:"  
    "mean_temperature",  
    inputs={  
        "observations":  
            "workflow:///prepare/outputs/observations.csv",  
    },  
    outputs={"result": "mean.json"},  
)
```

The function must be importable. Structured inputs and outputs become part of the task contract.

```
python tutorial/lesson_09_native_python_tasks.py
```

Run it now: the result is `{"mean_temperature_c": 13.0}`. No paid service or external network was required for any of the three structured tasks above.

Responsible AI--HPC Evidence

For an LLM-backed scientific step, preserve:

- prompt and staged context;
- provider, model, and parameters;
- raw response;
- parser or schema result;
- human review decision;
- governance basis and known limitations.

Provider success is not scientific validity. Never place secrets or sensitive data in exported workflow metadata.

AI--HPC Exercise Checkpoint

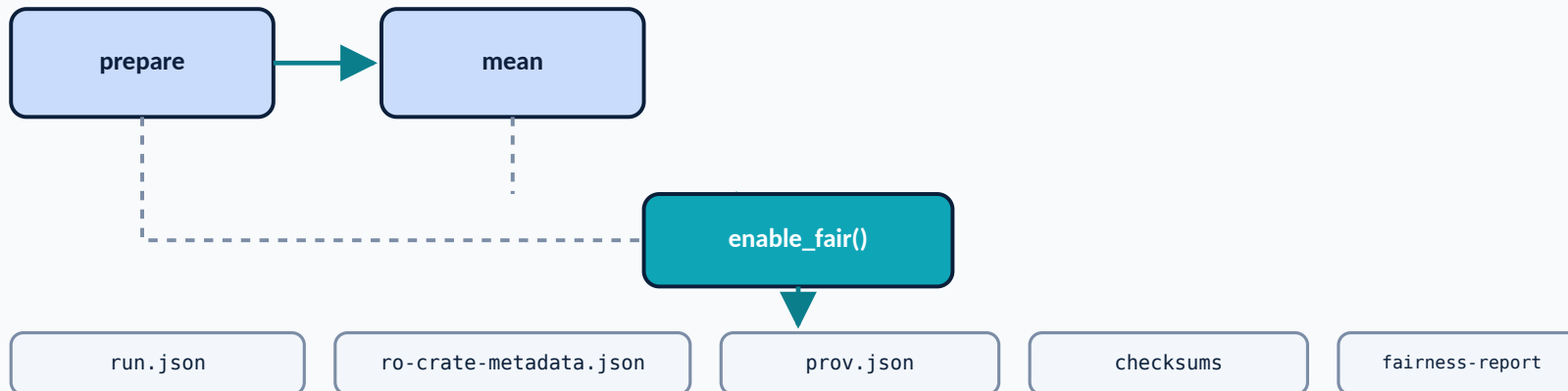
Connect one structured task to an upstream artifact:

1. Choose Native, LLM, or Web.
2. Name the upstream `workflow://` input.
3. Declare or identify the expected output.
4. Run the local mock or function.
5. Inspect both the scientific result and runtime metadata.

8. FAIR by Design: Provenance Recording as a Byproduct of the Graph

FAIR-by-Design Recording

```
workflow.enable_fair(FairProfile(title=..., license="Apache-2.0", creators=[...]))
prepare.declare_outputs(Artifact("outputs/observations.csv", ...))
```



No change to task execution: `enable_fair()` listens to the same lifecycle events lesson 08 uses and writes this bundle when the workflow ends. The dependency edge is inferred by DAGOn* itself (lesson 03), not separately declared.

```
python tutorial/lesson_15_fair_by_design.py
```

FAIR Checkpoint

Before publishing a workflow's results, confirm:

1. Every scientific artifact is declared, not just produced.
2. Creators, license, and access policy are recorded.
3. The dependency graph matches the provenance graph.
4. A machine-checkable fairness report accompanies the run.

Thank You

Scientific Workflows from Zero to Hero with DAGonStar

Dante Domizzi Sánchez-Gallegos

dantsanc@pa.uc3m.es

Contribute on GitHub: github.com/DagOnStar/dagonstar

Issues, pull requests, and new task-type backends are welcome.



Scan to contribute